الجمهورية الجزائرية الديمقراطية الشعبية
People's Democratic Republic of Algeria

Ministry of Higher Education and Scientific Research
National School of Cybersecurity

وزارة التعليم العالي والبحث العلمي
المدرسة الوطنية العليا في الأمن السيبراني

Department of Foundation Training

قسم التكوين القاعدي

Second semester Project: History of Algeria Database using Dynamic Data Structures

Level: 1st Year **Material:** Algorithms and Dynamic Data Structures

1. Introduction

This project aims at managing a database for personalities, events and dates related to the history of Algeria which are stocked into a text file. The personalities names and related information are separated by the "=" symbol, events by ":", and important dates are succeeded by {}.

The project focuses on implementing fundamental data structures, such as linked lists, stacks, queues, and trees using iterative and recursive modules. The user has a whole menu to manage the stocked data, which may invoke the following functions and procedures:

2. Modules for Linked lists and Queues

- TList* getPersonality(File *f): this function puts all personality names with their definitions into a linked list. Each node contains the full name and their definition.
- TList* getDatePersonality(File *f): this function puts all personality names with their dates of birth and death into a linked list.
- void getInfoByDates(TList *s, TList *DoB): this procedure takes a date of birth as input and returns the name and information of the corresponding personality.
- void getInfoByDates2(TList *s, TList *DoD): this procedure takes a date of death as input and returns the name and information of the corresponding personality.
- TList* sortWord(TList *syn): this function sorts the words alphabetically.
- TList* sortWord2(TList *syn): this function sorts ascendingly the list of nodes according to the number of chars.
- TList* sortPersonality(TList *syn): this function sorts ascendingly the list of nodes according to the age of personality.
- TList* deletepersonality(File *f, TList *s, TList *a, char *name): this function deletes a personality from the original file and from both linked lists.
- TList* updatePersonality(File *f, TList *s, TList *a, char *name, char *definition, char *DoB, char *DoD): this function updates a personality information; definition, date of birth, and date of death.
- TList* similarPersonality(TList *s, char *word): this function returns a list that contains all similar personalities in terms of year of birth or year of death.
- TList* countPersonality(TList *s, date *prt): this function returns a list that contains all the personalities where the date prt is included (birth, death or any other event).
- TList* palindromeName(TList *s): this function returns the list that contains all sorted palindromes names, where each word is inserted at the wright place to get the list sorted (names are extracted from the definition of personality or events such as city names, father names, etc.).
- TList* mergeNodes(TList *s, TList *a): this function merges the two nodes of the first and third lists (personality, and dates) into one node of a bidirectional list.
- TList* merge2Nodes(TList *s, TList *a): this function merges the two nodes of two previous lists into one node of a circular list.
- TList* addPersonality(TList *s, TList *a, char *name, char *DoB, char *DoD): this function adds a personality with name and dates into the first and last list, also in the text file.
- TList* addEvents(TList *b, char *namEvente, char *date): this function add an event with its date, to the second list, and also to the text file.
- TQueue* sName(TList *s): this function sorts the names according to the number of words of the name in a queue where each part is separated by a space.
- TQueue* ageP(TList *a): this function sorts the words according to their age.
- TQueue* toQueue(TList *merged): this function converts the list obtained from Merge function into a queue.

Second semester Project: History of Algeria Database using Dynamic Data Structures

Level: 1st Year **Material:** Algorithms and Dynamic Data Structures

3. Modules for Stacks

- TStack* toStack(TList *merged): this function converts the list returned by Merge function into a stack.
- TStack* getInfoPersonality(TStack *stk, char *name): this function takes a name as input and returns the definition, date of birth and date of death from the stack stk.
- TStack* sortNameStack(TStack *s): this function sorts the stack alphabetically.
- TStack* deleteName(TStack *stk, char *name): this function deletes a name from the stack.
- TStack* updateStack(TStack *stk, char *name, char *def, char *DoB, char *DoD): this function updates in the stack a personality name, definition, date of birth, date of death.
- TQueue* stackToQueue(TStack *stk): this function converts the stack resulted from the function toStack into a sorted queue.
- TList* stackToList(TStack *stk): this function converts the stack resulted from the function toStack into a bidirectional sorted linked list.
- TStack* addNameStack(TStack *stk, char *name, char *definition, char *DoB, char *DoD): this function adds a personality name with definition and dates into a sorted stack.
- TStack* definitionStack(TStack *stk): this function sorts the personality names according to the number of words in their definition, where each part is separated by a space.
- TStack* pronunciationStack(TStack *stk): this function sorts the events according to how they are described: short or long into two stacks.
- char* getSmallest(TStack *stk): this function returns the smallest definition the stack.
- void continuousSearch(TStack *stk): this procedure prints each continuous events where dates are overlapping.
- bool isPersonalityKilled(char *word): this function checks if a personality was killed (if it is mentioned in the definition that the personality was killed).
- TStack* recRevStack (TStack *stk): this function reverses a stack using recursion.

4. Modules for Binary Search Tree (BST)

- TTree* toTree(TStack *stk): this function converts a stack to a BST.
- TTree* fillTree(FILE *f): this function fills a BST by names from a file.
- TTree* getInfoNameTree(TTree *tr, char *name): this function takes a name as input and returns the definition and dates from the tree.
- TStack* addNameBST(TTree *tr, char *name, char *DoB, char *DoD): this function adds a word with dates of birth and death into a sorted tree.
- TTree* deleteNameBST(TTree *tr, char *name): this function deletes a name from the tree.
- TTree* updateNameBST(TTree *tr, char *name, char *s, char *DoB, char *DoD): this function updates a personality tree.
- TTree* traversalBSTinOrder(TTree *tr): this function performs an in-order traversal of the tree.
- TTree* traversalBSTpreOrder(TTree *tr): this function performs a pre-order traversal of the tree.
- TTree* traversalBSTpostOrder(TTree *tr): this function performs a post-order traversal of the tree.
- void heightSizeBST(TTree *tr): this procedure prints the height and size of the tree.
- TTree* lowestCommonAncestor(TTree *tr, char *word1, char *word2): this function returns the lowest common ancestor of two nodes (words).
- int countNodesRange(TTree *tr, int l, int h): this function counts the number of nodes that lie within a given range [l, h].

Second semester Project: History of Algeria Database using Dynamic Data Structures

Level: 1st Year **Material:** Algorithms and Dynamic Data Structures

- TTree* inOrderSuccessor(TTree *tr, char *word): this function returns the in-order successor of a given node in the tree.
- TTree* BSTMirror(TTree *tr): this function returns a mirror of the tree.
- bool isBalancedBST(TTree *tr): this function checks if a given tree is balanced.
- TTree* BTSMerge(TTree *tr1, TTree *tr2): this function merges two trees into a single balanced tree.

5. Module based on recursion

- int countOccurence(File *f, char *name): this function counts the number of occurrences of a name in the file using recursion.
- File* removeOccurence(File *f, char *word): this function removes all the occurrences of a name from the file using recursion.
- File* replaceOccurence(File *f, char *name, char *DoB, char *DoD): this recursive function replaces all the occurrences of a name from the file by the string name.
- void namePermutation(char *name): this procedure prints all the permutations of a given name using recursion.
- void subseqName(char *word): this recursive procedure generates all possible subsequences of a given event.
- void longestSubyear(char *date1, char *date2): this recursive procedure prints all the events that overlap with the mentioned dates.
- int distinctSubseqWord(char *event): this recursive function counts the number of distinct subsequences of the given event.
- bool isPalindromeWord(char *event): this recursive function checks if a given event overlap with a personality.

6. Graphical part

Develop a graphical user interface (GUI) in C. For this aim, you can use libraries such as:

- GTK+: a popular library for cross-platform GUIs.
- ncurses: a text-based UI for terminal applications.
- Raylib: a simple library for graphical applications.
- SDL (Simple DirectMedia Layer): a useful for rendering simple UI elements.

7. Report

All the students are requested to submit their complete project using C programming language, libraries, test suite, and project report by May 14th, 2026.

NB:

- The report must be clear, structured, and typed (PDF only).
- Handwritten submissions are not accepted.